

Neuro-Symbolic Text-to-Gloss

Manuale tecnico e operativo

Documentazione del progetto `neuro_symbolic_t2g`

Protocollo corrente Qwen2.5-0.5B

Come usare questo manuale

Questo testo insegna a comprendere, eseguire, analizzare ed estendere il progetto English-to-ASL-gloss. Le descrizioni operative derivano dalla configurazione corrente sotto **experiments/configs/qwen25-05b/**; non sostituiscono il controllo del file YAML risolto conservato con ogni run. Il manuale non dichiara risultati numerici della campagna corrente.

Indice

1	Panoramica e contratto scientifico	1
1.1	Cosa viene studiato	1
1.2	Tassonomia da non violare	1
1.3	Come leggere e riprodurre	1
2	Fondamenti	2
2.1	Modello autoregressivo	2
2.2	Supervisione, policy optimization e vincoli	2
2.3	Riproducibilità	2
3	Dati, split e prompting	3
3.1	Preprocessing ASLG-PC12	3
3.2	Vocabolario e anti-leakage	3
3.3	Prompt e retrieval TF-IDF	3
4	Modello, QLoRA e SFT	4
4.1	Configurazione esatta	4
4.2	Obiettivo e ricetta SFT	4
4.3	Fingerprint e riuso	4
5	GRPO e Dr-GRPO	5
5.1	Dr-GRPO e parametri correnti	5
6	Reward: definizione, qualificazione e limiti	6
6.1	Contratto comune e gate	6
6.2	Le cinque candidate esatte	6
6.3	Qualificazione congelata	6
6.4	Hard gate a due livelli	7
6.5	Copertura dei test borderline	7
7	Decodifica vincolata	8
7.1	Trie primario e maschera del vocabolario	8
7.2	PDA manuale	8
7.3	Massa mascherata	8
8	Benchmark structured-loss a stati ridotti	9
9	Valutazione e inferenza	10
9.1	Deployment e sampling	10
9.2	Metriche e interpretazione	10
9.3	Intervalli e confronti	10

9.4	Identità della cache	10
10	Disegno sperimentale canonico	11
10.1	Sette celle e dodici voci	11
10.2	Config e artifact	11
10.3	Ablazioni fuori matrice	11
11	Diagnostica congelata e decisioni	12
11.1	Rollout e reward	12
11.2	Markov, Viterbi hard e soft	12
12	Guida alla codebase ed estensioni	13
12.1	Mappa per directory e simboli	13
12.2	Ricette di estensione	13
12.3	Validazione	14
13	Troubleshooting, limiti e passi successivi	15
13.1	Diagnosi per sintomo	15
13.2	LEGACY — NON CONFRONTABILE	15
13.3	Passi successivi responsabili	15
14	Operazioni sul cluster	17
14.1	Setup online sul login node	17
14.2	Compute rigorosamente offline	17
14.3	Coda e campaign chain	17
15	API remota, TUI e monitoraggio	19
15.1	W&B offline e sicurezza	19
A	Comandi canonici e glossario	20
A.1	Comandi	20
A.2	Glossario	20

Capitolo 1

Panoramica e contratto scientifico

Il progetto traduce testo inglese in sequenze di glosse ASL. Una glossa è un’etichetta discreta associata a un segno, non una rappresentazione completa di spazio, simultaneità o componenti non manuali. Il verso è English-to-gloss sul corpus ASLG-PC12 [8, 4]; risultati gloss-to-English non sono confrontabili.

1.1 Cosa viene studiato

Il backbone è Qwen/Qwen2.5-0.5B-Instruct [11]. Si confrontano base, SFT, GRPO e SFT seguito da GRPO, con LoRA/QLoRA, retrieval TF-IDF e decodifica lessicalmente vincolata. “Neuro-simbolico” indica qui l’interazione fra distribuzione neurale e strutture discrete; non implica una grammatica ASL completa.

1.2 Tassonomia da non violare

Asse	Significato
Metodo	base, sft, grpo, sft-grpo: sistema appreso e inizializzazione.
Prompt	zero-shot o few-shot/retrieval: solo condizionamento in ingresso.
Variante	pda, hot, reward-*: ablazione controllata.
Lifecycle	train, baseline, ablation, probe: tipo di job/artifact.

Una modalità di prompt non è un metodo. Una baseline zero-shot è il metodo base non addestrato con quel prompt. Trie è il percorso primario; PDA è un’ablazione manuale. I probe congelati non sono obiettivi addestrabili.

1.3 Come leggere e riprodurre

I capitoli 3–8 spiegano dati e metodi, 9–11 protocollo e diagnostica, 12–15 codice e operazioni. Prima di una conclusione verificare config risolta, hash, split, checkpoint, modalità prompt, decoding e versione metrica. Questo manuale non inventa risultati correnti. docs/SOURCES.md è il registro completo delle fonti; la bibliografia finale contiene quelle citate nel testo.

Capitolo 2

Fondamenti

2.1 Modello autoregressivo

Dato un prompt x e una glossa tokenizzata y , il modello fattorizza $p_\theta(y \mid x) = \prod_t p_\theta(y_t \mid x, y_{<t})$. Il tokenizer opera su subtoken, mentre metriche e vocabolario di glosse operano soprattutto su token separati da spazi: confondere i due livelli produce gate e vincoli errati.

2.2 Supervisione, policy optimization e vincoli

SFT apprende dai target gold; GRPO confronta più completion dello stesso prompt; il constraint modifica il supporto disponibile in generazione. Sono interventi diversi: un Trie può garantire compatibilità lessicale senza migliorare il contenuto, e un reward ben formulato può essere empiricamente poco informativo. BLEU [9], ROUGE [6] e chrF misurano aspetti diversi e vanno letti insieme.

2.3 Riproducibilità

Un numero senza identità di dataset, modello, tokenizer, vocabolario, retrieval, grammar, decoding, seed e budget non è un risultato riproducibile. Il progetto materializza queste identità negli artifact e usa ID stabili per cache e bootstrap appaiato. Provenienza e failure espliciti sono parte del metodo.

Capitolo 3

Dati, split e prompting

3.1 Preprocessing ASLG-PC12

Il loader in `src/datasets/aslg_dataset.py` legge `text` e `gloss`, scarta righe vuote e deduplica prima dello split usando il testo inglese normalizzato (minuscole e spazi compressi). Dal train grezzo deriva uno split deterministico 90/10 con seed 42; un test grezzo eventuale non viene usato. Ogni riga riceve un ID SHA-256 del testo normalizzato e gold, e difficoltà per lunghezza: 1–5, 6–15, almeno 16 glosse.

3.2 Vocabolario e anti-leakage

`data/gloss_vocab.txt` è costruito soltanto dal train, ordinato in modo deterministico e accompagnato da sidecar; include BOS, EOS e UNK. Dev/test non definiscono lessico, archi o pesi. La matrice bigram è opzionale e diagnostica. La stessa separazione vale per l'indice retrieval.

3.3 Prompt e retrieval TF-IDF

`src/utils/prompting.py` usa il chat template del tokenizer, con fallback ChatML. Zero-shot contiene istruzione e sorgente; few-shot antepone esempi **English:/ASL gloss:.** Il retriever in `src/retrieval/example_retriever.py` usa TF-IDF, $k = 3$, soglia di autosimilarità 0.98 e cache `data/retriever_index`. Le dimostrazioni provengono solo dal train e il candidato corrente va escluso: controllare ID e hash evita self-retrieval e contaminazione. Nell'evaluator il nome CLI è `retrieval`; negli artifact è `few-shot`.

Capitolo 4

Modello, QLoRA e SFT

4.1 Configurazione esatta

La base usa Qwen2.5-0.5B-Instruct, sequenza massima 1024, pesi 4-bit, calcolo bfloat16, Unsloth e una GPU. LoRA modifica

$$W = W_0 + \frac{\alpha}{r}BA, \quad (4.1)$$

con $r = 32$, $\alpha = 64$, dropout 0.05, seed 3407 e target elencati in `base.yaml`: proiezioni q/k/v/o e gate/up/down [3]. Il backbone quantizzato resta congelato; gli adapter non sono per questo “4-bit”.

4.2 Obiettivo e ricetta SFT

$$\mathcal{L}_{\text{SFT}} = - \sum_{t=1}^T \log p_{\theta}(y_t \mid x, y_{<t}). \quad (4.2)$$

La config canonica `sft/zero-shot.yaml` usa 3 epoche, batch/device 4, accumulo 4, learning rate $2 \cdot 10^{-5}$, cosine scheduler, warmup 50, eval fraction 0.02 ed early stopping patience 3. SFT è addestrato zero-shot ma valutato con entrambi i prompt.

4.3 Fingerprint e riuso

Il checkpoint SFT è riusabile soltanto se fingerprint di config, modello, tokenizer, dataset/split e adapter coincidono. `sft-grpo` riusa l’adapter SFT; non va selezionato un checkpoint solo perché è il più recente. Conservare config risolta e origine, e usare `--resume` solo con la stessa identità.

Capitolo 5

GRPO e Dr-GRPO

Per ogni prompt vengono generate $G = 8$ completion. Concettualmente il confronto di gruppo centra i reward, $A_i = r_i - \bar{r}$. La formula spesso mostrata, $A_i = (r_i - \bar{r})/(s_r + \epsilon)$, *non* descrive questa config: in TRL 0.24.0 `scale_rewards:none` disattiva la divisione per deviazione standard, non il confronto nel gruppo [5, 12]. Gruppi a reward costante non danno discriminazione.

Con rapporto token $\rho_{it} = \pi_\theta / \pi_{\text{old}}$ e $\epsilon = 0.2$, il surrogate usa

$$\min(\rho_{it}A_i, \text{clip}(\rho_{it}, 1 - \epsilon, 1 + \epsilon)A_i). \quad (5.1)$$

Il KL verso la reference ha coefficiente $\beta = 0.04$; clipping rispetto alla policy dei rollout e ancoraggio alla reference hanno ruoli distinti.

5.1 Dr-GRPO e parametri correnti

`loss_type:dr_grpo` usa un denominatore globale costante invece della media per lunghezza della singola completion, schematicamente

$$\mathcal{L} = -\frac{1}{GL_{\max}} \sum_i \sum_{t \leq |y_i|} \mathcal{S}_{it}. \quad (5.2)$$

Questo riduce un bias, non garantisce lunghezze corrette [7]. Config: batch/device 1, accumulo 8, AdamW paged 8-bit, weight decay 0.1, grad norm 0.1, cosine, max prompt 256, $L_{\max} = 128$, temperatura 0.7, importance sampling token-level e completion troncate mascherate. `hot` cambia solo T a 1.3.

Capitolo 6

Reward: definizione, qualificazione e limiti

6.1 Contratto comune e gate

Esiste sempre *un solo* reward selezionato da `reward.name`, mai una somma pesata. La completion viene normalizzata con `extract_gloss_text`, poi Unicode `casefold` e `split`; la reference è testo piano e riceve `casefold+split`. Il gate richiede completion e reference non vuote e ogni token generato nel vocabolario. Un fallimento restituisce `invalid_score`, default `-1`. Il builder lo accetta soltanto finito e in $[-1, 1]$: NaN, infinito e valori esterni sono rifiutati. La similarità viene clampata in $[0, 1]$ e $R = 2s - 1$, quindi ogni reward è rigorosamente in $[-1, 1]$.

6.2 Le cinque candidate esatte

edit-validity

$s = 1 - d_{Lev}(g, y) / \max(|g|, |y|)$, Levenshtein sui token.

token-f1-validity

F1 dell'intersezione multinsieme con conteggi clipped; premia contenuto ma è intenzionalmente invariante al riordino.

chrFPP-validity

SacreBLEU chrF++: char order 6, word order 2, $\beta = 2$, whitespace disattivato e nessun epsilon smoothing.

rouge-l-validity

F1 dalla LCS sui token, senza stemming.

sbleu2-exp-validity

sentence BLEU-2, effective order, smoothing esponenziale e tokenizer `none` [10].

Correttezza della formula non equivale a utilità empirica. Edit penalizza ogni operazione uniformemente; multiset F1 ignora ordine; chrF++ tollera vicinanza di caratteri; ROUGE-L conserva ordine ma ammette sottosequenze; BLEU-2 è fragile su frasi corte, specialmente a una glossa.

6.3 Qualificazione congelata

Prima di un'ablazione si misurano tutte le candidate sullo stesso artifact con $G = 8$, retrieval, Trie, $T = 0.7$, massimo 128. Le soglie per candidata sono: gruppi a varianza zero ≤ 0.50 ,

all-min ≤ 0.50 , score unici medi ≥ 1.50 , best-minus-mean ≥ 0.05 , oltre a ranking, lunghezza e perturbazioni. Il report registra hash input/vocabolario, identità decoding, protocollo e reward, firme SacreBLEU e digest SHA-256 congiunto di `src/rewards/t2g_rewards.py` e `src/utls/text_utils.py`.

Le perturbazioni sono metric-specifiche: l'inversione per token-F1 è invarianza attesa e non fa gate. No-op, palindromi e casi non valutabili sono contati in `no_op_count` e `structurally_unevaluable_count`, non mascherati come successi o failure e non fanno gate. Degradazioni attese valutabili richiedono media non negativa e *evidenza positiva* di calo. È screening manuale, non prova di superiorità.

6.4 Hard gate a due livelli

`cluster/train.sh` richiede in `EXTRA_ARGS` l'opzione `--reward-qualification-report` per config reward-ablation. In modo indipendente, il trainer Python valida subito dopo il config e prima di risolvere run, dati o modello. Rifiuta report mancante/non JSON, protocollo stale, candidate fallita o inleggibile, mismatch di digest/impostazioni/SacreBLEU, gruppo, temperatura, prompt/retrieval, Trie o max length, artifact sorgente mancante/hash-alterato e vocabolario mancante, stale o non coincidente col config. L'invocazione Python diretta non aggira il gate; nessun gate promuove automaticamente una metrica.

```
python -m src.analysis rewards --config experiments/configs/qwen25-05b/probes/
rewards.yaml --input experiments/results/.../generations_x.json
EXTRA_ARGS="--reward-qualification-report experiments/analysis/qwen25-05b/rewards/
report.json" bash cluster/run_all.sh grpo-few-reward-token-f1
```

Le config sono manual-only. TUI/driver mostrano il campo report solo per train reward, accettano path repository-relative sotto `experiments/analysis/` e nella modalità train+eval inoltrano il report soltanto alla voce train.

6.5 Copertura dei test borderline

La suite copre vocabolario vuoto, gold vuoto o mancante, OOV e casefold Unicode. Copre inoltre spazi e punteggiatura, operazioni edit, duplicati e riordino, sequenze disgiunte, SacreBLEU a un token, bound stretti, chat TRL malformata o con final message, errori config e provenienza. Questi test provano invarianti e failure mode, non superiorità empirica.

Capitolo 7

Decodifica vincolata

7.1 Trie primario e maschera del vocabolario

Attenzione. Non attribuire al Trie il filtraggio di voci numeriche o subtoken. È GlossVocabularyMask a filtrare token ID secondo le proprie regole. Il Trie viene poi compilato sulle stringhe in `mask.vocab`, usando le codifiche normali e con spazio prefisso. Sono due stadi distinti.

Con storia h_t e insieme ammesso $A(h_t)$, il logits processor pone a $-\infty$ i logit fuori insieme e rinormalizza. Al termine di una glossa consente la successiva o EOS e gestisce separatori/newline. Il vincolo garantisce un percorso compilato, non correttezza semantica.

7.2 PDA manuale

`ablations/sft-grpo-zero-pda.yaml` abilita GrammarLLM PDA e lookahead. La grammatica LL(1) rappresenta prima glossa, coda e EOS; lo stack di parsing è diverso dal cammino prefisso del Trie. Il confronto mantiene fissi modello, inizializzazione, dati, prompt, seed e budget e richiede il gate sull'intero vocabolario reale. PDA non è il percorso di produzione né un quinto metodo.

7.3 Massa mascherata

Prima del masking,

$$M_t = \sum_{v \in A(h_t)} \text{softmax}(z_t)_v, \quad R_t = 1 - M_t. \quad (7.1)$$

Massa ammessa/rimossa ed entropia condizionata descrivono il conflitto tra policy e vincolo. Sono diagnostici congelati, non reward né obiettivi trainabili. Valori bassi di M_t non provano che il constraint corregga il significato.

Capitolo 8

Benchmark structured-loss a stati ridotti

È un benchmark isolato: congela Qwen, estrae il vettore dell'ultimo layer al confine assistant e allena soltanto una testa di emissione posizionale. Non è collegato al trainer SFT/GRPO, al driver o alla TUI.

Lo spazio ha le 512 glosse train più frequenti più <OTHER>; BOS/EOS esistono solo nel grafo. Sequenze oltre 64 sono escluse e registrate, mai troncate. Il grafo usa $\alpha = 0.1$ e scala 0.25; dev/test non definiscono stati o archi. Dichiarando $y_0 = \text{BOS}$, lo score comprende l'arco EOS finale:

$$S(y; x, L) = \sum_{t=1}^L e_t(y_t; x) + \sum_{t=1}^L a(y_{t-1}, y_t) + a(y_L, \text{EOS}). \quad (8.1)$$

Con $\log Z = \log \sum_{|z|=L} \exp S(z; x, L)$, l'implementazione riporta

$$\text{NLL} = (\log Z - S)/L, \quad (8.2)$$

non la somma non normalizzata. La partizione termina esattamente a L .

Le braccia sono CE indipendente, uniform-support, true-weights e shuffled-weights a supporto fisso. Feature e righe sono condivise; argmax posizionale a lunghezza gold non è Viterbi. Le metriche di estrazione stanno nel manifest separato dalle metriche delle teste. La resource gate segnala pass/fail ma non promuove automaticamente: true-weights deve battere i controlli su tutti i seed configurati e la decisione resta manuale. Avvio offline:

```
sbatch cluster/structured_probe.sh
```

Capitolo 9

Valutazione e inferenza

9.1 Deployment e sampling

Su 2.000 esempi test seeded, ogni prompt ha una completion `deployment greedy (do_sample=false)` e $N = 5$ draw `sampling` indipendenti a $T = 0.7$. Le headline sono solo greedy; `sampling` misura qualità attesa, diversità/headroom e Pass@k. Un oracle scelto col gold è non deployable e va separato. I seed per sample derivano deterministicamente da seed di protocollo, ID stabile sorgente+gold, mode e indice: `batching`, `cache` e `resume` non li cambiano.

9.2 Metriche e interpretazione

Si riportano corpus BLEU-4 e chrF2, ROUGE-L medio, micro token-F1, exact match normalizzato, edit similarity, validità e lunghezze. Il successo Pass@k richiede ROUGE-L ≥ 0.3 e validità lessicale; lo stimatore è $1 - \binom{n-c}{k} / \binom{n}{k}$ [2]. Includere errore di lunghezza signed/assoluto, ratio, empty e truncation-hit. Validità non è qualità; una decodifica vincolata può aumentare la prima e peggiorare le altre metriche.

9.3 Intervalli e confronti

Il bootstrap è deterministico e clustered per prompt. Le metriche sentence dei draw sono prima aggregate nel prompt; BLEU/chrF corpus vengono ricalcolati a ogni resample. Un delta appaiato richiede gli stessi ID unici nello stesso ordine e gli stessi indici bootstrap. Non usare intervalli indipendenti per sostituire il test appaiato.

9.4 Identità della cache

Cache e baseline esplicita devono coincidere su versione metrica, hash degli ID, dataset, modello, tokenizer, vocabolario, retrieval, grammar, decoding, seed, mode, N , budget e temperatura. Ogni mismatch fallisce esplicitamente. Gli artifact conservano `decoding_mode`, `completion_index`, raw generations, metriche, config, checkpoint e firme SacreBLEU.

Capitolo 10

Disegno sperimentale canonico

10.1 Sette celle e dodici voci

Le sette celle sono due baseline eval-only (base zero/few), SFT zero-shot e il 2×2 GRPO: inizializzazione base/SFT per prompt train zero/few. La coda ha 12 voci: 2 eval baseline più train+eval per le 5 celle apprese. Ogni eval appresa esegue zero-shot e retrieval. Non contare le due gambe come metodi diversi.

Metodo	Inizializzazione	Prompt train
SFT	base	zero-shot
GRPO	base	zero-shot
GRPO	base	few-shot
SFT-GRPO	SFT	zero-shot
SFT-GRPO	SFT	few-shot

10.2 Config e artifact

La radice è `experiments/configs/qwen25-05b/`; `base.yaml` è ricetta condivisa, non cella. I runnable dichiarano `model_tag`, `method`, `train_prompt_mode`, `variant` e `kind`. Checkpoint/log seguono

```
experiments/{checkpoints,logs}/qwen25-05b/<method>/<train-prompt>/[ablations/<variant>/]run_<timestamp>/
```

e risultati

```
experiments/results/qwen25-05b/<method>/<train-prompt>/[ablations/<variant>/]eval-<prompt>/run_<timestamp>/
```

Le baseline usano `baseline/<prompt>/run_*`. Identità config e artifact devono concordare; non copiare YAML in gerarchie parallele.

10.3 Ablazioni fuori matrice

PDA, hot e quattro reward alternativi sono manuali; GRPO few-shot è il controllo edit-validity. Rollout/reward/Markov sono probe su generazioni congelate. Structured-loss è un benchmark head-only. Nessuno entra nella campagna primaria senza nuova preregistrazione.

Capitolo 11

Diagnostica congelata e decisioni

11.1 Rollout e reward

I probe consumano `generations_*.json`; non cercano input e non generano automaticamente. Rollout raggruppa per ID, controlla gold coerente e misura varianza, saturazione, unicità, best-minus-mean, correlazioni e bucket di lunghezza. Le boundary del reward sono quelle del [chapter 6](#); superarle autorizza solo l'ispezione/ablazione manuale.

11.2 Markov, Viterbi hard e soft

Il bigram $O(L)$ può usare il lessico completo; hard/soft usano piccoli grafi e falliscono prima dell'allocazione oltre `max_states=256`. Il Viterbi hard massimizza transizioni logaritmiche e applica $-\lambda \mathbf{1}[i = j]$ soltanto alle transizioni *interiori*; l'arco finale verso EOS non riceve la penalità di self-loop. Soft sostituisce max con log-sum-exp temperata. Si analizzano Spearman, discriminazione pairwise, lunghezza e correlazione parziale. In formula, solo per i passi interni,

$$\delta_t(j) = \max_i \{ \delta_{t-1}(i) + \log P_{ij} - \lambda \mathbf{1}[i = j] \}; \quad (11.1)$$

la chiusura usa $\max_i \{ \delta(i) + \log P_{i,\text{EOS}} \}$ senza penalità.

Non è una Differentiable Viterbi Layer: non vi sono emissioni apprese, backpointer differenziabili o gradienti [1]. La decisione è se il segnale aggiunga ranking oltre edit/bigram sui medesimi sample; anche in caso positivo serve un'ablazione di training isolata per una pretesa causale.

```
INPUT=experiments/results/.../generations_x.json sbatch cluster/probe.sh rollouts
experiments/configs/qwen25-05b/probes/rollouts.yaml
INPUT=experiments/results/.../generations_x.json sbatch cluster/probe.sh markov
experiments/configs/qwen25-05b/probes/markov.yaml
```


Capitolo 12

Guida alla codebase ed estensioni

12.1 Mappa per directory e simboli

`src/datasets/`

prepara righe, split, matrici e grafi strutturati.

`src/models/`

`model_loader.py` carica Qwen/adapter; `structured_gloss_head.py` implementa la testa ridotta.

`src/training/`

entry point SFT, GRPO, eval e benchmark strutturato.

`src/grammar/`

implementa logits processor Trie/PDA e telemetria.

`src/rewards/t2g_rewards.py`

registry, normalizzazione, gate e metriche reward.

`src/retrieval/`

indice e ricerca TF-IDF.

`src/analysis/`

rollout, reward, Markov e structured benchmark.

`src/utils/`

config, path/identity, prompt, metriche, cache e monitor.

`experiments/configs/`

ricette canoniche; `cluster/` launcher offline/chain; `remote/` API e TUI; `tests/` invarianti.

12.2 Ricette di estensione

Nuova config. Estendere la base, cambiare un asse, assegnare identità univoca, aggiungere validazione/path test e registrarla nel driver solo se deve essere esposta. Non confondere prompt e method.

Nuovo reward. Implementare similarità pura, riusare normalizzazione, gate, clamp e transform, registrare un solo nome, aggiungere borderline e perturbation contract, config manuale e qualificazione con digest/hash.

Nuovo probe. Consumare artifact congelato esplicito, versionare schema/protocollo, separare diagnostica da training e definire boundary prima di guardare i risultati.

Nuovo backend retrieval. Implementare build/query, fingerprint di corpus e parametri, esclusione self-match, cache fail-closed e test anti-leakage.

Nuova route/API. Aggiungere modello request, autenticazione, validazione repository-relative, operazione idempotente e test API/TUI; non considerare SQLite fonte autoritativa.

12.3 Validazione

La suite comprende config, dataset, prompt/retrieval, reward, grammar, metriche, training/eval, cache/path, launcher offline, probe, API e TUI. Comandi canonici:

```
uv run python -m pytest tests/ -v
python tests/validate_configs.py
```

Uno skip GPU/cache è un prerequisito assente, non un pass. Registrare comando, versioni, pass/fail/skip e non riportare conteggi storici come correnti.

Capitolo 13

Troubleshooting, limiti e passi successivi

13.1 Diagnosi per sintomo

Sintomo	Azione
Config/identity mismatch	Stampare config risolta; verificare method, prompt, variant, kind e checkpoint source; non rinominare directory a mano.
Cache/model offline mancante	Non abilitare rete nel job: rieseguire setup sul login node, quindi preflight.
Output vuoti o tutti invalidi	Ispezionare chat extraction, EOS, vocabolario, OOV e massa ammessa; distinguere gate reward da Trie.
Reward senza varianza	Controllare gruppi, gold, $G = 8$, gate e distribuzione; non lanciare training per “vedere se migliora”.
OOM	Verificare config effettiva, batch/accumulo, max length, quantizzazione e processi concorrenti; non cambiare più assi nell’esperimento.
Eval non appaiabile	Rigenerare con stessi ID ordinati e identica protocol identity; non forzare la cache.
PDA fallisce	Eseguire preflight con PDA=1 sul vocabolario completo.
W&B non visibile	Il compute è offline: preservare directory e sincronizzare dopo, da nodo con rete.

13.2 LEGACY — NON CONFRONTABILE

Attenzione. Artifact, metriche o figure anteriori al protocollo canonico `qwen25-05b`, incluse versioni metriche precedenti, sono LEGACY E NON CONFRONTABILI. Non unirli a tabelle correnti.

Questo manuale omette valori storici: non sono evidenza della campagna attuale.

13.3 Passi successivi responsabili

Direzioni possibili, non implementazioni presenti: valutazione umana/linguistica ASL; più seed e potenza; retrieval appreso con anti-leakage; reward d’ordine qualificato e isolato; transizioni

condizionate alla sorgente; vera testa strutturata con Viterbi e gradient-flow test; studio causale della massa rimossa. Ogni estensione richiede ipotesi, controllo, failure mode, test, budget e criterio di decisione preregistrati. Superare un probe non dimostra beneficio di training.

Capitolo 14

Operazioni sul cluster

14.1 Setup online sul login node

```
ssh <user>@gcluster.dmi.unict.it
cd ~/neuro_symbolic_t2g
bash cluster/setup.sh
```

Il setup online segue sempre il percorso canonico login node, **srun**, compute node, **apptainer****run--nv** con **/shared/sifs/latest.sif**. Il login node non cerca né esegue Apptainer. Lo script richiede la storica allocazione con account e partition **thesis-course**, QoS **gpu-xlarge**, una GPU, shard 22000, 48G e 8 CPU. Python non viene mai eseguito sul login node. Il setup installa in **~/.local** senza alterare lo stack torch/CUDA, verifica versioni, scarica Qwen e ASLG-PC12 e costruisce vocabolario/sidecar. **BUILD_BIGRAM=1** abilita la matrice opzionale. Le versioni presidiate includono Transformers 5.3.0, TRL 0.24.0, PEFT 0.19.1, Unsloth 2026.7.1 e torchao 0.16–0.17.

14.2 Compute rigorosamente offline

Preflight, train, eval e probe esportano prima di Python:

```
HF_HUB_OFFLINE=1
TRANSFORMERS_OFFLINE=1
HF_DATASETS_OFFLINE=1
WANDB_MODE=offline
WANDB_DISABLE_WEAVE=true
WANDB_SILENT=true
```

Il preflight controlla import, snapshot, dataset, vocab, retrieval/bigram se richiesti e W&B. Non scarica e non allena.

14.3 Coda e campaign chain

La QoS ammette un solo job submitted/attivo per utente: anche **pip-reset/setup**, che usa **srun**, va eseguito soltanto quando non vi sono altre allocazioni attive. Il setup richiede inoltre rete disponibile nell’allocazione. **run_all.sh** mantiene una chain sequenziale di 12 voci. Non lanciare una seconda campagna in parallelo. Comandi:

```
CONFIG=experiments/configs/qwen25-05b/sft-grpo/zero-shot.yaml sbatch cluster/
    preflight.sh
bash cluster/run_all.sh
```

```
CONFIG=experiments/configs/qwen25-05b/sft-grpo/zero-shot.yaml EXTRA_ARGS="--resume"
sbatch cluster/train.sh
```

Usare gli strumenti `chain-show/monitor` per osservare l'avanzamento. Una failure va corretta e ripresa con la stessa identità, non aggirata cambiando path. PDA e reward usano i gate aggiuntivi descritti nei capitoli 6 e 7.

Capitolo 15

API remota, TUI e monitoraggio

Il servizio FastAPI in `remote/` esegue via SSH tick idempotenti. Lo stato autoritativo è `.chain_state/` sul cluster; SQLite remoto è solo `cache/journal`. Pause è soft e non cancella il job attivo. Tutte le route tranne `GET/` richiedono `X-Auth-Token`.

```
uv run --extra dev uvicorn app:app --port 8000 --app-dir remote
uv run --extra tui python remote/tui.py --url http://localhost:8000 --token local-token
```

Il registry espone celle canoniche e varianti manuali. Il campo storico API `ablation:true` seleziona/sostituisce la coda predefinita, non include automaticamente PDA/hot. Per reward train il report qualificato è obbligatorio; eval-only non deve riceverlo. Rollout e Markov non sono queue config; il benchmark structured è assente da driver e TUI. Un 401 indica autenticazione, mentre un 502 segnala tipicamente un problema SSH/cluster; lo status può restituire cache marcata non raggiungibile.

15.1 W&B offline e sicurezza

I compute scrivono run W&B offline in directory isolate per gamba eval. Il sync è successivo da ambiente con rete [13]. Non mettere token HF, W&B, API o chiavi SSH in repository, YAML, log o command history condivisa; preferire environment/secret store, permessi minimi e rotazione. Esporre l'API solo su interfaccia fidata o tunnel SSH, usare token forte e non stampare header. Validare ogni path come repository-relative e non accettare argomenti shell arbitrari dalla TUI.

Da Windows gli script `sync_cluster.ps1` scaricano log, checkpoint e risultati. Conservare la gerarchia timestampata: appiattirla perde provenienza.

Appendice A

Comandi canonici e glossario

A.1 Comandi

Usare i path completi nei file di job; i segnaposto vanno sostituiti senza cambiare l'identità sperimentale.

```
# online: login -> srun -> compute -> apptainer run
# QoS: eseguire senza altre allocazioni attive
bash cluster/setup.sh

# compute offline
CONFIG=experiments/configs/qwen25-05b/sft-grpo/zero-shot.yaml sbatch cluster/
    preflight.sh
bash cluster/run_all.sh
CONFIG=experiments/configs/qwen25-05b/sft/zero-shot.yaml sbatch cluster/train.sh
CONFIG=experiments/configs/qwen25-05b/baseline/few-shot.yaml sbatch cluster/eval.sh

# manuali
PDA=1 CONFIG=experiments/configs/qwen25-05b/ablations/sft-grpo-zero-pda.yaml sbatch
    cluster/preflight.sh
bash cluster/run_all.sh sft-grpo-zero-pda
bash cluster/run_all.sh sft-grpo-zero-hot

# validazione locale
uv run python -m pytest tests/ -v
python tests/validate_configs.py

# documento
latexmk -pdf main.tex
```

A.2 Glossario

Artifact identity

Config, modello/checkpoint, dati, tokenizer, prompt, constraint, seed, budget, hash e versioni che rendono interpretabile un output.

ASL gloss

Etichetta testuale semplificata di un segno, non lingua completa.

ASLG-PC12

Corpus parallelo del progetto.

Deployment

Una completion greedy per prompt; sorgente delle headline.

Diagnostico congelato

Analisi senza gradiente su generazioni esistenti.

Dr-GRPO

Loss con denominatore globale per ridurre length bias.

Few-shot/retrieval

Modalità di prompt con esempi TF-IDF, non metodo.

GRPO

Policy optimization relativa a gruppi di completion.

LoRA/QLoRA

Adapter low-rank su backbone congelato/quantizzato.

Metodo

Base, SFT, GRPO o SFT-GRPO.

PDA

Automa a pila GrammarLLM, ablazione manuale.

Prompt-clustered bootstrap

Resampling dei prompt mantenendo uniti i draw.

Reward

Singolo segnale scalare usato per aggiornare la policy.

Sampling

Cinque draw eval a $T = 0.7$; la qualificazione reward ne richiede otto.

SFT

Fine-tuning supervisionato sui target gold.

Structured NLL

NLL con partizione a lunghezza gold fissa; qui $(\log Z - S)/L$.

Trie

Albero di prefissi compilato dal vocabolario mascherato; percorso primario.

Validità

Appartenenza lessicale/formale, distinta dalla qualità traduttiva.

Viterbi

DP hard/soft qui diagnostica a piccoli stati, non DVL addestrabile.

Zero-shot

Modalità di prompt senza esempi, non metodo.

Bibliografia

- [1] Anonymous. ViterbiPlanNet: Differentiable viterbi planning. *arXiv preprint arXiv:2603.04265*, 2026. Attribuzione per possibile lavoro futuro; non descrive i probe correnti.
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [4] Hugging Face. achrafothman/aslg_pc12 dataset card. https://huggingface.co/datasets/achrafothman/aslg_pc12. Consultato come sorgente del dataset.
- [5] Hugging Face. TRL GRPOTrainer documentation. https://huggingface.co/docs/trl/main/en/grpo_trainer.
- [6] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. *Text Summarization Branches Out*, 2004.
- [7] Zichen Liu et al. Understanding R1-Zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025. Riferimento del progetto per Dr.GRPO e length bias.
- [8] Achraf Othman and Mohamed Jemni. English-ASL gloss parallel corpus 2012: ASLG-PC12. *arXiv preprint arXiv:1112.0168*, 2012.
- [9] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *ACL*, 2002.
- [10] Matt Post. A call for clarity in reporting BLEU scores. In *Proceedings of WMT*, 2018.
- [11] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [12] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [13] Weights & Biases. Offline run mode. <https://docs.wandb.ai/guides/runs/run-modes/>.